SETTING UP THE MOCK PROVIDER

The next step that you need to do is to define the mock provider that Pact will use. Remember that when it comes to contract testing, the consumer doesn't directly interact with the provider, but instead interacts with the mock provider that Pact provides. To do that, add the following lines of code to your consumer-contract.spec.js file:

```
const provider = new PactV3({
  dir: path.resolve(process.cwd(), 'pacts'),
  consumer: 'WebConsumer',
  provider: 'MoviesAPI',
});
```

This code calls the `PactV3` constructor to create a new mock provider that we can use as part of our consumer contract test. As part of the constructor, we're passing parameters to define the directory for the contracts to be stored in the pacts folder of the current working directory. This will also create the folder if it doesn't exist yet. We also pass in the parameters to define the name of the consumer and the provider, which, in this example, are `'WebConsumer'` and `'MoviesAPI'`. These names will be used throughout the contract generation and when you upload the contract to a Pact Broker.

REGISTERING THE CONSUMER EXPECTATION

Let's now structure the first test by using Jest's describe block. If you're not familiar with Jest (https://jestjs.io/), Jest is a popular JavaScript testing framework that comes built-in with all the methods and utilities you need for testing your JavaScript code. Jest should have been installed in your local project as part of the project's requirements. If you haven't installed the GitHub project's dependencies, then make sure to do it by running `npm install`.

Now, add the following code into the consumer-contract.spec.js file after the provider configuration: